DBMS - ADVANCED SQL QUERY IMPLEMENTATION

Project Tiltle : Lenskart E-Commerce Database

TASK 1: IMPLEMENT SUBQUERIES AND NESTED QUERIES

1.1 Products with Price Greater Than 1500

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM LenskartProduct
  3  WHERE Price > (SELECT 1500 FROM DUAL);

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------
         2
Blue Light Glasses
      1800

         4
Classic Black Sunglasses
      2000

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------
```

1.2 Customers Who Have Placed Orders

```
SQL> SELECT Customer_ID, Customer_Name
  2  FROM LenskartCustomer
  3  WHERE Customer_ID IN
  4        (SELECT Customer_ID FROM LenskartOrder);

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
          1
Arun Kumar

          2
Priya Sharma

          3
Rahul Kumar


CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
          4
Divya Raj

          5
Karthik S
```

TASK 2: FIND PRODUCTS ABOVE AVERAGE PRICE

```
SQL> SELECT Product_ID, Product_Name, Brand, Price
  2  FROM LenskartProduct
  3  WHERE Price > (SELECT AVG(Price) FROM LenskartProduct)
  4  ORDER BY Price DESC;
```

```
PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
BRAND
--------------------------------------------------------------------------------
     PRICE
----------
         4
Classic Black Sunglasses
Fastrack
      2000


PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
BRAND
--------------------------------------------------------------------------------
     PRICE
----------
         2
Blue Light Glasses
John Jacobs
      1800


PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
BRAND
--------------------------------------------------------------------------------
     PRICE
----------
         3
Round Metal Frame
Lenskart Air
      1500
```

TASK 3: IDENTIFY CUSTOMERS WITH MAXIMUM PURCHASES

```
SQL> SELECT C.Customer_ID,
  2         C.Customer_Name,
  3         SUM(O.Total_Amount) AS Total_Purchase
  4  FROM LenskartCustomer C
  5  JOIN LenskartOrder O
  6  ON C.Customer_ID = O.Customer_ID
  7  GROUP BY C.Customer_ID, C.Customer_Name
  8  HAVING SUM(O.Total_Amount) =
  9         (SELECT MAX(Total_Purchase)
 10          FROM
 11          (SELECT SUM(Total_Amount) AS Total_Purchase
 12           FROM LenskartOrder
 13           GROUP BY Customer_ID))
 14  ORDER BY Total_Purchase DESC;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_PURCHASE
--------------
         4
Divya Raj
         2000
```

TASK 4: GENERATE COMPLEX BUSINESS QUERIES

4.1 Customers Who Purchased Above Average Order Amount

```
SQL> SELECT C.Customer_ID,
  2         C.Customer_Name,
  3         O.Order_ID,
  4         O.Total_Amount
  5  FROM LenskartCustomer C
  6  JOIN LenskartOrder O
  7  ON C.Customer_ID = O.Customer_ID
  8  WHERE O.Total_Amount >
  9        (SELECT AVG(Total_Amount)
 10         FROM LenskartOrder)
 11  ORDER BY O.Total_Amount DESC;
```

```
CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
  ORDER_ID TOTAL_AMOUNT
---------- ------------
         4
Divya Raj
         4         2000

         2
Priya Sharma
         2         1800

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
  ORDER_ID TOTAL_AMOUNT
---------- ------------

         3
Rahul Kumar
         3         1500
```

4.2 Products Sold in Orders Above Average Order Value

```
SQL> SELECT DISTINCT P.Product_ID,
  2         P.Product_Name,
  3         P.Price
  4  FROM LenskartProduct P
  5  JOIN LenskartOrderItem OI
  6  ON P.Product_ID = OI.Product_ID
  7  WHERE OI.Order_ID IN
  8        (SELECT Order_ID
  9         FROM LenskartOrder
 10         WHERE Total_Amount >
 11               (SELECT AVG(Total_Amount)
 12                FROM LenskartOrder));
```

```
PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------
         2
Blue Light Glasses
      1800

         3
Round Metal Frame
      1500

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------

         4
Classic Black Sunglasses
      2000
```

4.3 Products with Sales Quantity Greater Than Average

```
SQL> SELECT P.Product_ID,
  2         P.Product_Name,
  3         SUM(OI.Quantity) AS Total_Quantity_Sold
  4  FROM LenskartProduct P
  5  JOIN LenskartOrderItem OI
  6  ON P.Product_ID = OI.Product_ID
  7  GROUP BY P.Product_ID, P.Product_Name
  8  HAVING SUM(OI.Quantity) >
  9         (SELECT AVG(Quantity)
 10          FROM LenskartOrderItem)
 11  ORDER BY Total_Quantity_Sold DESC;

no rows selected
```

TASK 5: PREPARE AN ADVANCED SQL REPORT

```
SQL> SELECT C.Customer_ID,
  2         C.Customer_Name,
  3         COUNT(O.Order_ID) AS Total_Orders,
  4         SUM(O.Total_Amount) AS Total_Purchase,
  5         ROUND(AVG(O.Total_Amount), 2) AS Average_Order_Amount,
  6         MAX(O.Total_Amount) AS Highest_Order
  7  FROM LenskartCustomer C
  8  JOIN LenskartOrder O
  9  ON C.Customer_ID = O.Customer_ID
 10  GROUP BY C.Customer_ID, C.Customer_Name
 11  HAVING SUM(O.Total_Amount) >=
 12         (SELECT AVG(Total_Amount)
 13          FROM LenskartOrder)
 14  ORDER BY Total_Purchase DESC;
```

```
CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_ORDERS TOTAL_PURCHASE AVERAGE_ORDER_AMOUNT HIGHEST_ORDER
------------ -------------- -------------------- -------------
           4
Divya Raj
            1           2000                 2000          2000

           2
Priya Sharma
            1           1800                 1800          1800

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_ORDERS TOTAL_PURCHASE AVERAGE_ORDER_AMOUNT HIGHEST_ORDER
------------ -------------- -------------------- -------------

           3
Rahul Kumar
            1           1500                 1500          1500
```